VLLM专题（十九）—兼容 OpenAI 的服务器

vLLM 提供了一个 HTTP 服务器，能够实现 OpenAI 的 Completions API、Chat API 等功能！

您可以通过 `vllm serve` 命令启动服务器，或者通过 Docker 启动：

```
vllm serve NousResearch/Meta-Llama-3-8B-Instruct --dtype auto --api-key token-abc123
```

要调用服务器，您可以使用官方的 OpenAI Python 客户端，或任何其他 HTTP 客户端。

```
from openai import OpenAI
client = OpenAI(
    base_url="http://localhost:8000/v1",
    api_key="token-abc123",
)

completion = client.chat.completions.create(
  model="NousResearch/Meta-Llama-3-8B-Instruct",
  messages=[
    {
            "role": "user", "content": "Hello!"}
  ]
)

print(completion.choices[0].message)
```

1. 支持的 API

我们目前支持以下OpenAI API：

生成API（Completions API）(/v1/completions)
仅适用于文本生成模型（–task generate）。
注意：不支持`suffix`参数。

聊天生成API（Chat Completions API）(/v1/chat/completions)
仅适用于带有聊天模板的文本生成模型（–task generate）。
注意：`parallel_tool_calls`和`user`参数被忽略。

嵌入API（Embeddings API）(/v1/embeddings)
仅适用于嵌入模型（–task embed）。

转录API（Transcriptions API）(/v1/audio/transcriptions)
仅适用于自动语音识别（ASR）模型（OpenAI Whisper）（–task generate）。

此外，我们还提供以下自定义API：

分词API（Tokenizer API）(/tokenize, /detokenize)
适用于任何带有分词器的模型。

池化API（Pooling API）(/pooling)
适用于所有池化模型。

评分API（Score API）(/score)
适用于嵌入模型和交叉编码模型（–task score）。

重排名API（Re-rank API）(/rerank, /v1/rerank, /v2/rerank)
实现了Jina AI的v1重排名API。
也兼容Cohere的v1和v2重排名API。
Jina和Cohere的API非常相似；Jina的重排名端点响应中包含额外的信息。
仅适用于交叉编码模型（–task score）。

2. 聊天模板

为了使语言模型支持聊天协议，vLLM要求模型在其分词器配置中包含一个聊天模板。聊天模板是一个Jinja2模板，指定了在输入中如何编码角色、消息和其他特定于聊天的令牌。

以 **NousResearch/Meta-Llama-3-8B-Instruct** 为例的聊天模板可以在这里找到。

有些模型虽然进行了指令/聊天微调，但并没有提供聊天模板。对于这些模型，你可以在`--chat-template`参数中手动指定它们的聊天模板，提供聊天模板的文件路径或字符串形式的模板。如果没有聊天模板，服务器将无法处理聊天请求，所有聊天请求都会报错。

```
vllm serve <model> --chat-template ./path-to-chat-template.jinja
```

vLLM社区为流行模型提供了一套聊天模板。你可以在**examples**目录下找到它们。

随着多模态聊天API的引入，OpenAI规范现在接受一种新格式的聊天消息，该格式同时指定了消息的类型和文本字段。下面提供了一个示例：

```
completion = client.chat.completions.create(
  model="NousResearch/Meta-Llama-3-8B-Instruct",
  messages=[
    {
            "role": "user", "content": [{
            "type": "text", "text": "Classify this sentiment: vLLM is wonderful!"}]}
  ]
)
```

大多数LLM的聊天模板期望`content`字段是一个字符串，但像 **meta-llama/Llama-Guard-3-1B** 这样的新模型期望`content`根据请求中的OpenAI模式进行格式化。vLLM提供了最佳努力的支持来自动检测这一点，检测结果会以类似“检测到聊天模板内容格式为…”的字符串记录，并会将传入的请求内部转换为匹配检测到的格式，格式可以是以下之一：

> **“string”**：一个字符串。
>
> 示例：`"Hello world"`
>
> **“openai”**：一个字典列表，类似于OpenAI模式。
>
> 示例：`[{"type": "text", "text": "Hello world!"}]`

如果结果不是你所期望的，你可以通过设置`--chat-template-content-format` CLI参数来覆盖使用的格式。

3. 额外参数

vLLM支持一组不属于OpenAI API的一般参数。为了使用这些参数，你可以将它们作为额外的参数传递到OpenAI客户端中，或者如果你是直接使用HTTP调用，可以将它们直接合并到JSON负载中。

例如：

```
completion = client.chat.completions.create(
  model="NousResearch/Meta-Llama-3-8B-Instruct",
  messages=[
    {
            "role": "user", "content": "Classify this sentiment: vLLM is wonderful!"}
  ],
  extra_body={

    "guided_choice": ["positive", "negative"]
  }
)
```

在这个例子中，`extra_body`包含了一个额外的参数`"guided_choice"`，它指定了情感分类的选项，模型将根据这个额外参数来生成结果。

4. 额外HTTP头

目前仅支持`X-Request-Id` HTTP请求头。可以通过`--enable-request-id-headers`启用此功能。

请注意，启用这些头部可能会在高QPS（每秒请求数）情况下显著影响性能。由于这个原因，我们建议在路由层（例如通过Istio）实现HTTP头部，而不是在vLLM层中实现。有关更多详细信息，请参阅此PR（拉取请求）。

示例：

```
completion = client.chat.completions.create(
  model="NousResearch/Meta-Llama-3-8B-Instruct",
  messages=[
    {
            "role": "user", "content": "Classify this sentiment: vLLM is wonderful!"}
  ],
  extra_headers={

    "x-request-id": "sentiment-classification-00001",
  }
)
print(completion._request_id)

completion = client.completions.create(
  model="NousResearch/Meta-Llama-3-8B-Instruct",
  prompt="A robot may not injure a human being",
  extra_headers={

    "x-request-id": "completion-test",
  }
)
print(completion._request_id)
```

在这个示例中，`extra_headers` 包含了额外的 `x-request-id` 请求头，用于标识请求。模型生成的请求ID将被打印出来。

5. CLI参考

vllm serve 解释

vllm serve 命令用于启动与OpenAI兼容的服务器。

```
usage: vllm serve [-h] [--host HOST] [--port PORT]
                  [--uvicorn-log-level {
            debug,info,warning,error,critical,trace}]
                  [--allow-credentials] [--allowed-origins ALLOWED_ORIGINS]
                  [--allowed-methods ALLOWED_METHODS]
                  [--allowed-headers ALLOWED_HEADERS] [--api-key API_KEY]
                  [--lora-modules LORA_MODULES [LORA_MODULES ...]]
                  [--prompt-adapters PROMPT_ADAPTERS [PROMPT_ADAPTERS ...]]
                  [--chat-template CHAT_TEMPLATE]
                  [--chat-template-content-format {
            auto,string,openai}]
                  [--response-role RESPONSE_ROLE] [--ssl-keyfile SSL_KEYFILE]
                  [--ssl-certfile SSL_CERTFILE] [--ssl-ca-certs SSL_CA_CERTS]
                  [--enable-ssl-refresh] [--ssl-cert-reqs SSL_CERT_REQS]
                  [--root-path ROOT_PATH] [--middleware MIDDLEWARE]
                  [--return-tokens-as-token-ids]
                  [--disable-frontend-multiprocessing]
                  [--enable-request-id-headers] [--enable-auto-tool-choice]
                  [--tool-call-parser {
            granite-20b-fc,granite,hermes,internlm,jamba,llama3_json,mistral,pythonic} or
 name registered in --tool-parser-plugin]
                  [--tool-parser-plugin TOOL_PARSER_PLUGIN] [--model MODEL]
                  [--task {
            auto,generate,embedding,embed,classify,score,reward,transcription}]
                  [--tokenizer TOKENIZER] [--hf-config-path HF_CONFIG_PATH]
                  [--skip-tokenizer-init] [--revision REVISION]
                  [--code-revision CODE_REVISION]
                  [--tokenizer-revision TOKENIZER_REVISION]
                  [--tokenizer-mode {
            auto,slow,mistral,custom}]
                  [--trust-remote-code]
                  [--allowed-local-media-path ALLOWED_LOCAL_MEDIA_PATH]
                  [--download-dir DOWNLOAD_DIR]
                  [--load-format {
            auto,pt,safetensors,npcache,dummy,tensorizer,sharded_state,gguf,bitsandbytes,
mistral,runai_streamer}]
                  [--config-format {
            auto,hf,mistral}]
                  [--dtype {
```

```
            auto,half,float16,bfloat16,float,float32}]
                  [--kv-cache-dtype {
            auto,fp8,fp8_e5m2,fp8_e4m3}]
                  [--max-model-len MAX_MODEL_LEN]
                  [--guided-decoding-backend GUIDED_DECODING_BACKEND]
                  [--logits-processor-pattern LOGITS_PROCESSOR_PATTERN]
                  [--model-impl {
            auto,vllm,transformers}]
                  [--distributed-executor-backend {
            ray,mp,uni,external_launcher}]
                  [--pipeline-parallel-size PIPELINE_PARALLEL_SIZE]
                  [--tensor-parallel-size TENSOR_PARALLEL_SIZE]
                  [--enable-expert-parallel]
                  [--max-parallel-loading-workers MAX_PARALLEL_LOADING_WORKERS]
                  [--ray-workers-use-nsight] [--block-size {
            8,16,32,64,128}]
                  [--enable-prefix-caching | --no-enable-prefix-caching]
                  [--disable-sliding-window] [--use-v2-block-manager]
                  [--num-lookahead-slots NUM_LOOKAHEAD_SLOTS] [--seed SEED]
                  [--swap-space SWAP_SPACE] [--cpu-offload-gb CPU_OFFLOAD_GB]
                  [--gpu-memory-utilization GPU_MEMORY_UTILIZATION]
                  [--num-gpu-blocks-override NUM_GPU_BLOCKS_OVERRIDE]
                  [--max-num-batched-tokens MAX_NUM_BATCHED_TOKENS]
                  [--max-num-partial-prefills MAX_NUM_PARTIAL_PREFILLS]
                  [--max-long-partial-prefills MAX_LONG_PARTIAL_PREFILLS]
                  [--long-prefill-token-threshold LONG_PREFILL_TOKEN_THRESHOLD]
                  [--max-num-seqs MAX_NUM_SEQS] [--max-logprobs MAX_LOGPROBS]
                  [--disable-log-stats]
                  [--quantization {
            aqlm,awq,deepspeedfp,tpu_int8,fp8,ptpc_fp8,fbgemm_fp8,modelopt,nvfp4,marlin,g
guf,gptq_marlin_24,gptq_marlin,awq_marlin,gptq,compressed-tensors,bitsandbytes,qqq,hqq,ex
perts_int8,neuron_quant,ipex,quark,moe_wna16,None}]
                  [--rope-scaling ROPE_SCALING] [--rope-theta ROPE_THETA]
                  [--hf-overrides HF_OVERRIDES] [--enforce-eager]
                  [--max-seq-len-to-capture MAX_SEQ_LEN_TO_CAPTURE]
                  [--disable-custom-all-reduce]
                  [--tokenizer-pool-size TOKENIZER_POOL_SIZE]
                  [--tokenizer-pool-type TOKENIZER_POOL_TYPE]
                  [--tokenizer-pool-extra-config TOKENIZER_POOL_EXTRA_CONFIG]
                  [--limit-mm-per-prompt LIMIT_MM_PER_PROMPT]
                  [--mm-processor-kwargs MM_PROCESSOR_KWARGS]
                  [--disable-mm-preprocessor-cache] [--enable-lora]
                  [--enable-lora-bias] [--max-loras MAX_LORAS]
                  [--max-lora-rank MAX_LORA_RANK]
                  [--lora-extra-vocab-size LORA_EXTRA_VOCAB_SIZE]
                  [--lora-dtype {
            auto,float16,bfloat16}]
                  [--long-lora-scaling-factors LONG_LORA_SCALING_FACTORS]
```

```
                [--long-lora-scaling-factors LONG_LORA_SCALING_FACTORS]
                [--max-cpu-loras MAX_CPU_LORAS] [--fully-sharded-loras]
                [--enable-prompt-adapter]
                [--max-prompt-adapters MAX_PROMPT_ADAPTERS]
                [--max-prompt-adapter-token MAX_PROMPT_ADAPTER_TOKEN]
                [--device {
          auto,cuda,neuron,cpu,openvino,tpu,xpu,hpu}]
                [--num-scheduler-steps NUM_SCHEDULER_STEPS]
                [--use-tqdm-on-load | --no-use-tqdm-on-load]
                [--multi-step-stream-outputs [MULTI_STEP_STREAM_OUTPUTS]]
                [--scheduler-delay-factor SCHEDULER_DELAY_FACTOR]
                [--enable-chunked-prefill [ENABLE_CHUNKED_PREFILL]]
                [--speculative-model SPECULATIVE_MODEL]
                [--speculative-model-quantization {
          aqlm,awq,deepspeedfp,tpu_int8,fp8,ptpc_fp8,fbgemm_fp8,modelopt,nvfp4,marlin,g
guf,gptq_marlin_24,gptq_marlin,awq_marlin,gptq,compressed-tensors,bitsandbytes,qqq,hqq,ex
perts_int8,neuron_quant,ipex,quark,moe_wna16,None}]
                [--num-speculative-tokens NUM_SPECULATIVE_TOKENS]
                [--speculative-disable-mqa-scorer]
                [--speculative-draft-tensor-parallel-size SPECULATIVE_DRAFT_TENSOR_PARA
LLEL_SIZE]
                [--speculative-max-model-len SPECULATIVE_MAX_MODEL_LEN]
                [--speculative-disable-by-batch-size SPECULATIVE_DISABLE_BY_BATCH_SIZE]
                [--ngram-prompt-lookup-max NGRAM_PROMPT_LOOKUP_MAX]
                [--ngram-prompt-lookup-min NGRAM_PROMPT_LOOKUP_MIN]
                [--spec-decoding-acceptance-method {
          rejection_sampler,typical_acceptance_sampler}]
                [--typical-acceptance-sampler-posterior-threshold TYPICAL_ACCEPTANCE_SA
MPLER_POSTERIOR_THRESHOLD]
                [--typical-acceptance-sampler-posterior-alpha TYPICAL_ACCEPTANCE_SAMPLE
R_POSTERIOR_ALPHA]
                [--disable-logprobs-during-spec-decoding [DISABLE_LOGPROBS_DURING_SPEC_
DECODING]]
                [--model-loader-extra-config MODEL_LOADER_EXTRA_CONFIG]
                [--ignore-patterns IGNORE_PATTERNS]
                [--preemption-mode PREEMPTION_MODE]
                [--served-model-name SERVED_MODEL_NAME [SERVED_MODEL_NAME ...]]
                [--qlora-adapter-name-or-path QLORA_ADAPTER_NAME_OR_PATH]
                [--show-hidden-metrics-for-version SHOW_HIDDEN_METRICS_FOR_VERSION]
                [--otlp-traces-endpoint OTLP_TRACES_ENDPOINT]
                [--collect-detailed-traces COLLECT_DETAILED_TRACES]
                [--disable-async-output-proc]
                [--scheduling-policy {
          fcfs,priority}]
                [--scheduler-cls SCHEDULER_CLS]
                [--override-neuron-config OVERRIDE_NEURON_CONFIG]
                [--override-pooler-config OVERRIDE_POOLER_CONFIG]
                [--compilation-config COMPILATION_CONFIG]
```

```
                [--kv-transfer-config KV_TRANSFER_CONFIG]
                [--worker-cls WORKER_CLS]
                [--worker-extension-cls WORKER_EXTENSION_CLS]
                [--generation-config GENERATION_CONFIG]
                [--override-generation-config OVERRIDE_GENERATION_CONFIG]
                [--enable-sleep-mode] [--calculate-kv-scales]
                [--additional-config ADDITIONAL_CONFIG] [--enable-reasoning]
                [--reasoning-parser {
           deepseek_r1}] [--disable-log-requests]
                [--max-log-len MAX_LOG_LEN] [--disable-fastapi-docs]
                [--enable-prompt-tokens-details]
                [--enable-server-load-tracking]
```

6. 配置文件

你可以通过YAML配置文件加载CLI参数。参数名称必须是上述命令行参数的长格式。

例如，下面是一个YAML配置文件的示例：

```yaml
# config.yaml

host: "127.0.0.1"
port: 6379
uvicorn-log-level: "info"
```

要使用上述配置文件，你可以运行以下命令：

```
vllm serve SOME_MODEL --config config.yaml
```

> **注意**
> 如果同时通过命令行和配置文件提供了某个参数，命令行中的值将优先使用。优先级顺序为：命令行 > 配置文件中的值 > 默认值。

7. API参考

7.1 生成API（Completions API）

我们的生成API与OpenAI的生成API兼容；你可以使用官方的OpenAI Python客户端与之进行交互。

代码示例：`openai_completion_client.py`

```python
# SPDX-License-Identifier: Apache-2.0

from openai import OpenAI

# Modify OpenAI's API key and API base to use vLLM's API server.
openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    # defaults to os.environ.get("OPENAI_API_KEY")
    api_key=openai_api_key,
    base_url=openai_api_base,
)

models = client.models.list()
model = models.data[0].id

# Completion API
stream = False
completion = client.completions.create(
    model=model,
    prompt="A robot may not injure a human being",
    echo=False,
    n=2,
    stream=stream,
    logprobs=3)

print("Completion results:")
if stream:
    for c in completion:
        print(c)
else:
    print(completion)
```

额外参数

以下是支持的采样参数。

```
use_beam_search: bool = False
top_k: Optional[int] = None
min_p: Optional[float] = None
repetition_penalty: Optional[float] = None
length_penalty: float = 1.0
stop_token_ids: Optional[list[int]] = Field(default_factory=list)
include_stop_str_in_output: bool = False
ignore_eos: bool = False
min_tokens: int = 0
skip_special_tokens: bool = True
spaces_between_special_tokens: bool = True
truncate_prompt_tokens: Optional[Annotated[int, Field(ge=1)]] = None
allowed_token_ids: Optional[list[int]] = None
prompt_logprobs: Optional[int] = None
```

以下是支持的额外参数：

```
add_special_tokens: bool = Field(
    default=True,
    description=(
        "If true (the default), special tokens (e.g. BOS) will be added to "
        "the prompt."),
)
response_format: Optional[ResponseFormat] = Field(
    default=None,
    description=
    ("Similar to chat completion, this parameter specifies the format of "
     "output. Only {'type': 'json_object'}, {'type': 'json_schema'} or "
     "{'type': 'text' } is supported."),
)
guided_json: Optional[Union[str, dict, BaseModel]] = Field(
    default=None,
    description="If specified, the output will follow the JSON schema.",
)
guided_regex: Optional[str] = Field(
    default=None,
    description=(
        "If specified, the output will follow the regex pattern."),
)
guided_choice: Optional[list[str]] = Field(
    default=None,
    description=(
        "If specified, the output will be exactly one of the choices."),
)
guided_grammar: Optional[str] = Field(
    default=None,
    description=(
        "If specified, the output will follow the context-free grammar.")
```

```
        "If specified, the output will follow the context free grammar."),
    )
    guided_decoding_backend: Optional[str] = Field(
        default=None,
        description=(
            "If specified, will override the default guided decoding backend "
            "of the server for this specific request. If set, must be one of "
            "'outlines' / 'lm-format-enforcer'"))
    guided_whitespace_pattern: Optional[str] = Field(
        default=None,
        description=(
            "If specified, will override the default whitespace pattern "
            "for guided json decoding."))
    priority: int = Field(
        default=0,
        description=(
            "The priority of the request (lower means earlier handling; "
            "default: 0). Any priority other than 0 will raise an error "
            "if the served model does not use priority scheduling."))
    logits_processors: Optional[LogitsProcessors] = Field(
        default=None,
        description=(
            "A list of either qualified names of logits processors, or "
            "constructor objects, to apply when sampling. A constructor is "
            "a JSON object with a required 'qualname' field specifying the "
            "qualified name of the processor class/factory, and optional "
            "'args' and 'kwargs' fields containing positional and keyword "
            "arguments. For example: {'qualname': "
            "'my_module.MyLogitsProcessor', 'args': [1, 2], 'kwargs': "
            "{'param': 'value'}}."))
    return_tokens_as_token_ids: Optional[bool] = Field(
        default=None,
        description=(
            "If specified with 'logprobs', tokens are represented "
            " as strings of the form 'token_id:{token_id}' so that tokens "
            "that are not JSON-encodable can be identified."))
```

7.2 聊天API

我们的聊天API与OpenAI的聊天生成API兼容；你可以使用官方的OpenAI Python客户端与之进行交互。

我们支持与视觉和音频相关的参数；有关更多信息，请参阅我们的多模态输入指南。

注意：不支持 `image_url.detail` 参数。

代码示例：`openai_chat_completion_client.py`

```python
# SPDX-License-Identifier: Apache-2.0

from openai import OpenAI

# Modify OpenAI's API key and API base to use vLLM's API server.
openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    # defaults to os.environ.get("OPENAI_API_KEY")
    api_key=openai_api_key,
    base_url=openai_api_base,
)

models = client.models.list()
model = models.data[0].id

chat_completion = client.chat.completions.create(
    messages=[{

        "role": "system",
        "content": "You are a helpful assistant."
    }, {

        "role": "user",
        "content": "Who won the world series in 2020?"
    }, {

        "role":
        "assistant",
        "content":
        "The Los Angeles Dodgers won the World Series in 2020."
    }, {

        "role": "user",
        "content": "Where was it played?"
    }],
    model=model,
)

print("Chat completion results:")
print(chat_completion)
```

额外参数

以下是支持的采样参数：

```
  best_of: Optional[int] = None
  use_beam_search: bool = False
  top_k: Optional[int] = None
  min_p: Optional[float] = None
  repetition_penalty: Optional[float] = None
  length_penalty: float = 1.0
  stop_token_ids: Optional[list[int]] = Field(default_factory=list)
  include_stop_str_in_output: bool = False
  ignore_eos: bool = False
  min_tokens: int = 0
  skip_special_tokens: bool = True
  spaces_between_special_tokens: bool = True
  truncate_prompt_tokens: Optional[Annotated[int, Field(ge=1)]] = None
  prompt_logprobs: Optional[int] = None
```

以下是支持的额外参数：

```
  echo: bool = Field(
    default=False,
    description=(
        "If true, the new message will be prepended with the last message "
        "if they belong to the same role."),
)
add_generation_prompt: bool = Field(
    default=True,
    description=
    ("If true, the generation prompt will be added to the chat template. "
     "This is a parameter used by chat template in tokenizer config of the "
     "model."),
)
continue_final_message: bool = Field(
    default=False,
    description=
    ("If this is set, the chat will be formatted so that the final "
     "message in the chat is open-ended, without any EOS tokens. The "
     "model will continue this message rather than starting a new one. "
     "This allows you to \"prefill\" part of the model's response for it. "
     "Cannot be used at the same time as `add_generation_prompt`."),
)
add_special_tokens: bool = Field(
    default=False,
    description=(
        "If true, special tokens (e.g. BOS) will be added to the prompt "
        "on top of what is added by the chat template. "
        "For most models, the chat template takes care of adding the "
        "special tokens so this should be set to false (as is the "
        "default)."),
)
```

```
)
documents: Optional[list[dict[str, str]]] = Field(
    default=None,
    description=
    ("A list of dicts representing documents that will be accessible to "
     "the model if it is performing RAG (retrieval-augmented generation)."
     " If the template does not support RAG, this argument will have no "
     "effect. We recommend that each document should be a dict containing "
     "\"title\" and \"text\" keys."),
)
chat_template: Optional[str] = Field(
    default=None,
    description=(
        "A Jinja template to use for this conversion. "
        "As of transformers v4.44, default chat template is no longer "
        "allowed, so you must provide a chat template if the tokenizer "
        "does not define one."),
)
chat_template_kwargs: Optional[dict[str, Any]] = Field(
    default=None,
    description=("Additional kwargs to pass to the template renderer. "
                 "Will be accessible by the chat template."),
)
mm_processor_kwargs: Optional[dict[str, Any]] = Field(
    default=None,
    description=("Additional kwargs to pass to the HF processor."),
)
guided_json: Optional[Union[str, dict, BaseModel]] = Field(
    default=None,
    description=("If specified, the output will follow the JSON schema."),
)
guided_regex: Optional[str] = Field(
    default=None,
    description=(
        "If specified, the output will follow the regex pattern."),
)
guided_choice: Optional[list[str]] = Field(
    default=None,
    description=(
        "If specified, the output will be exactly one of the choices."),
)
guided_grammar: Optional[str] = Field(
    default=None,
    description=(
        "If specified, the output will follow the context free grammar."),
)
guided_decoding_backend: Optional[str] = Field(
    default=None,
```

```
    description=(
        "If specified, will override the default guided decoding backend "
        "of the server for this specific request. If set, must be either "
        "'outlines' / 'lm-format-enforcer'"))
guided_whitespace_pattern: Optional[str] = Field(
    default=None,
    description=(
        "If specified, will override the default whitespace pattern "
        "for guided json decoding."))
priority: int = Field(
    default=0,
    description=(
        "The priority of the request (lower means earlier handling; "
        "default: 0). Any priority other than 0 will raise an error "
        "if the served model does not use priority scheduling."))
request_id: str = Field(
    default_factory=lambda: f"{
            random_uuid()}",
    description=(
        "The request_id related to this request. If the caller does "
        "not set it, a random_uuid will be generated. This id is used "
        "through out the inference process and return in response."))
logits_processors: Optional[LogitsProcessors] = Field(
    default=None,
    description=(
        "A list of either qualified names of logits processors, or "
        "constructor objects, to apply when sampling. A constructor is "
        "a JSON object with a required 'qualname' field specifying the "
        "qualified name of the processor class/factory, and optional "
        "'args' and 'kwargs' fields containing positional and keyword "
        "arguments. For example: {'qualname': "
        "'my_module.MyLogitsProcessor', 'args': [1, 2], 'kwargs': "
        "{'param': 'value'}}."))
return_tokens_as_token_ids: Optional[bool] = Field(
    default=None,
    description=(
        "If specified with 'logprobs', tokens are represented "
        " as strings of the form 'token_id:{token_id}' so that tokens "
        "that are not JSON-encodable can be identified."))
```

7.3 嵌入API

我们的嵌入API与OpenAI的嵌入API兼容；你可以使用官方的OpenAI Python客户端与之进行交互。

如果模型具有聊天模板，你可以使用消息列表替换输入（与聊天API相同的模式），这些消息将作为模型的单个提示进行处理。

代码示例：`openai_embedding_client.py`

```python
# SPDX-License-Identifier: Apache-2.0

from openai import OpenAI

# Modify OpenAI's API key and API base to use vLLM's API server.
openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    # defaults to os.environ.get("OPENAI_API_KEY")
    api_key=openai_api_key,
    base_url=openai_api_base,
)

models = client.models.list()
model = models.data[0].id

responses = client.embeddings.create(
    input=[
        "Hello my name is",
        "The best thing about vLLM is that it supports many different models"
    ],
    model=model,
)

for data in responses.data:
    print(data.embedding)  # List of float of len 4096
```

7.3.1 多模态输入

你可以通过为服务器定义一个自定义聊天模板，并在请求中传递消息列表，向嵌入模型传递多模态输入。

7.3.1.1 VLM2Vec

要启动模型服务：

```
vllm serve TIGER-Lab/VLM2Vec-Full --task embed \
  --trust-remote-code --max-model-len 4096 --chat-template examples/template_vlm2vec.jinj
a
```

> **重要提示**
> 由于 **VLM2Vec** 和 **Phi-3.5-Vision** 具有相同的模型架构，我们必须显式地传递 `--task embed` 来运行此模型的嵌入模式，而不是文本生成模式。

该自定义聊天模板与此模型的原始模板完全不同，可以在这里找到：`examples/template_vlm2vec.jinja`

由于请求模式并没有由OpenAI客户端定义，我们使用更低级的 `requests` 库向服务器发送请求。

```
import requests

image_url = "https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-wisconsin-madison-the-nature-boardwalk.jpg/2560px-Gfp-wisconsin-madison-the-nature-boardwalk.jpg"

response = requests.post(
    "http://localhost:8000/v1/embeddings",
    json={

        "model": "TIGER-Lab/VLM2Vec-Full",
        "messages": [{

            "role": "user",
            "content": [
                {
            "type": "image_url", "image_url": {
            "url": image_url}},
                {
            "type": "text", "text": "Represent the given image."},
            ],
        }],
        "encoding_format": "float",
    },
)
response.raise_for_status()
response_json = response.json()
print("Embedding output:", response_json["data"][0]["embedding"])
```

完整示例如下：

```
# SPDX-License-Identifier: Apache-2.0

import argparse
import base64
import io

import requests
from PIL import Image

image_url = "https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-wisconsin-madison-the-nature-boardwalk.jpg/2560px-Gfp-wisconsin-madison-the-nature-boardwalk.jpg"
```

```python
def vlm2vec():
    response = requests.post(
        "http://localhost:8000/v1/embeddings",
        json={

            "model":
            "TIGER-Lab/VLM2Vec-Full",
            "messages": [{

                "role":
                "user",
                "content": [
                    {

                        "type": "image_url",
                        "image_url": {

                            "url": image_url
                        }
                    },
                    {

                        "type": "text",
                        "text": "Represent the given image."
                    },
                ],
            }],
            "encoding_format":
            "float",
        },
    )
    response.raise_for_status()
    response_json = response.json()

    print("Embedding output:", response_json["data"][0]["embedding"])


def dse_qwen2_vl(inp: dict):
    # Embedding an Image
    if inp["type"] == "image":
        messages = [{

            "role":
            "user",
            "content": [{

                "type": "image_url",
```

```
            "image_url": {

                "url": inp["image_url"],
            }
        }, {

            "type": "text",
            "text": "What is shown in this image?"
        }]
    }]
# Embedding a Text Query
else:
    # MrLight/dse-qwen2-2b-mrl-v1 requires a placeholder image
    # of the minimum input size
    buffer = io.BytesIO()
    image_placeholder = Image.new("RGB", (56, 56))
    image_placeholder.save(buffer, "png")
    buffer.seek(0)
    image_placeholder = base64.b64encode(buffer.read()).decode('utf-8')
    messages = [{

        "role":
        "user",
        "content": [
            {

                "type": "image_url",
                "image_url": {

                    "url": f"{
          image_placeholder}",
                }
            },
            {

                "type": "text",
                "text": f"Query: {
          inp['content']}"
            },
        ]
    }]

response = requests.post(
    "http://localhost:8000/v1/embeddings",
    json={

        "model": "MrLight/dse-qwen2-2b-mrl-v1",
```

```
            "messages": messages,
            "encoding_format": "float",
        },
    )
    response.raise_for_status()
    response_json = response.json()

    print("Embedding output:", response_json["data"][0]["embedding"])


if __name__ == '__main__':
    parser = argparse.ArgumentParser(
        "Script to call a specified VLM through the API. Make sure to serve "
        "the model with --task embed before running this.")
    parser.add_argument("--model",
                        type=str,
                        choices=["vlm2vec", "dse_qwen2_vl"],
                        required=True,
                        help="Which model to call.")
    args = parser.parse_args()

    if args.model == "vlm2vec":
        vlm2vec()
    elif args.model == "dse_qwen2_vl":
        dse_qwen2_vl({

            "type": "image",
            "image_url": image_url,
        })
        dse_qwen2_vl({

            "type": "text",
            "content": "What is the weather like today?",
        })
```

7.3.1.2 DSE-Qwen2-MRL

要启动模型服务：

```
vllm serve MrLight/dse-qwen2-2b-mrl-v1 --task embed \
  --trust-remote-code --max-model-len 8192 --chat-template examples/template_dse_qwen2_vl
.jinja
```

重要提示
与 **VLM2Vec** 相似，我们必须显式地传递 `--task embed`。
此外，**MrLight/dse-qwen2-2b-mrl-v1** 需要一个用于嵌入的 **EOS** 令牌，这由一个自定义聊天模板处理：`examples/template_dse_qwen2_vl.jinja`。

重要提示
MrLight/dse-qwen2-2b-mrl-v1 需要一个最小图像尺寸的占位符图像，用于文本查询嵌入。有关详细信息，请参阅下面的完整代码示例。

完整示例如下：`openai_chat_embedding_client_for_multimodal.py`

```
# SPDX-License-Identifier: Apache-2.0

import argparse
import base64
import io

import requests
from PIL import Image

image_url = "https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-wisconsin-madi
son-the-nature-boardwalk.jpg/2560px-Gfp-wisconsin-madison-the-nature-boardwalk.jpg"


def vlm2vec():
    response = requests.post(
        "http://localhost:8000/v1/embeddings",
        json={

            "model":
            "TIGER-Lab/VLM2Vec-Full",
            "messages": [{

                "role":
                "user",
                "content": [
                    {

                        "type": "image_url",
                        "image_url": {

                            "url": image_url
                        }
                    },
```

```
                    {

                        "type": "text",
                        "text": "Represent the given image."
                    },
                ],
            }],
            "encoding_format":
            "float",
        },
    )
    response.raise_for_status()
    response_json = response.json()

    print("Embedding output:", response_json["data"][0]["embedding"])


def dse_qwen2_vl(inp: dict):
    # Embedding an Image
    if inp["type"] == "image":
        messages = [{

            "role":
            "user",
            "content": [{

                "type": "image_url",
                "image_url": {

                    "url": inp["image_url"],
                }
            }, {

                "type": "text",
                "text": "What is shown in this image?"
            }]
        }]
    # Embedding a Text Query
    else:
        # MrLight/dse-qwen2-2b-mrl-v1 requires a placeholder image
        # of the minimum input size
        buffer = io.BytesIO()
        image_placeholder = Image.new("RGB", (56, 56))
        image_placeholder.save(buffer, "png")
        buffer.seek(0)
        image_placeholder = base64.b64encode(buffer.read()).decode('utf-8')
        messages = [{
```

```
            "role":
            "user",
            "content": [
                {

                    "type": "image_url",
                    "image_url": {

                        "url": f"{
              image_placeholder}",
                    }
                },
                {

                    "type": "text",
                    "text": f"Query: {
              inp['content']}"
                },
            ]
        }]

    response = requests.post(
        "http://localhost:8000/v1/embeddings",
        json={

            "model": "MrLight/dse-qwen2-2b-mrl-v1",
            "messages": messages,
            "encoding_format": "float",
        },
    )
    response.raise_for_status()
    response_json = response.json()

    print("Embedding output:", response_json["data"][0]["embedding"])


if __name__ == '__main__':
    parser = argparse.ArgumentParser(
        "Script to call a specified VLM through the API. Make sure to serve "
        "the model with --task embed before running this.")
    parser.add_argument("--model",
                        type=str,
                        choices=["vlm2vec", "dse_qwen2_vl"],
                        required=True,
                        help="Which model to call.")
    args = parser.parse_args()
```

```
if args.model == "vlm2vec":
    vlm2vec()
elif args.model == "dse_qwen2_vl":
    dse_qwen2_vl({

        "type": "image",
        "image_url": image_url,
    })
    dse_qwen2_vl({

        "type": "text",
        "content": "What is the weather like today?",
    })
```

7.3.2 额外参数

以下是支持的池化参数：

```
    additional_data: Optional[Any] = None
```

默认支持以下额外参数：

```
    add_special_tokens: bool = Field(
        default=True,
        description=(
            "If true (the default), special tokens (e.g. BOS) will be added to "
            "the prompt."),
    )
    priority: int = Field(
        default=0,
        description=(
            "The priority of the request (lower means earlier handling; "
            "default: 0). Any priority other than 0 will raise an error "
            "if the served model does not use priority scheduling."))
```

对于类似聊天的输入（即传递了 `messages`），支持以下额外参数：

```
    add_special_tokens: bool = Field(
        default=False,
        description=(
            "If true, special tokens (e.g. BOS) will be added to the prompt "
            "on top of what is added by the chat template. "
            "For most models, the chat template takes care of adding the "
            "special tokens so this should be set to false (as is the "
            "default)."),
    )
    chat_template: Optional[str] = Field(
        default=None,
        description=(
            "A Jinja template to use for this conversion. "
            "As of transformers v4.44, default chat template is no longer "
            "allowed, so you must provide a chat template if the tokenizer "
            "does not define one."),
    )
    chat_template_kwargs: Optional[dict[str, Any]] = Field(
        default=None,
        description=("Additional kwargs to pass to the template renderer. "
                     "Will be accessible by the chat template."),
    )
    mm_processor_kwargs: Optional[dict[str, Any]] = Field(
        default=None,
        description=("Additional kwargs to pass to the HF processor."),
    )
    priority: int = Field(
        default=0,
        description=(
            "The priority of the request (lower means earlier handling; "
            "default: 0). Any priority other than 0 will raise an error "
            "if the served model does not use priority scheduling."))
```

7.4 转录API

我们的转录API与OpenAI的转录API兼容；你可以使用官方的OpenAI Python客户端与之进行交互。

> **注意**
> 要使用转录API，请通过以下命令安装包含额外音频依赖项的vLLM：

```
pip install vllm[audio]
```

示例代码如下：`openai_transcription_client.py`

```
# SPDX-License-Identifier: Apache-2.0
import asyncio
```

```python
import json

import httpx
from openai import OpenAI

from vllm.assets.audio import AudioAsset

mary_had_lamb = AudioAsset('mary_had_lamb').get_local_path()
winning_call = AudioAsset('winning_call').get_local_path()

# Modify OpenAI's API key and API base to use vLLM's API server.
openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"
client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)


def sync_openai():
    with open(str(mary_had_lamb), "rb") as f:
        transcription = client.audio.transcriptions.create(
            file=f,
            model="openai/whisper-small",
            language="en",
            response_format="json",
            temperature=0.0)
        print("transcription result:", transcription.text)


sync_openai()


# OpenAI Transcription API client does not support streaming.
async def stream_openai_response():
    data = {

        "language": "en",
        'stream': True,
        "model": "openai/whisper-large-v3",
    }
    url = openai_api_base + "/audio/transcriptions"
    print("transcription result:", end=' ')
    async with httpx.AsyncClient() as client:
        with open(str(winning_call), "rb") as f:
            async with client.stream('POST', url, files={
            'file': f},
                                     data=data) as response:
```

```
                async for line in response.aiter_lines():
                    # Each line is a JSON object prefixed with 'data: '
                    if line:
                        if line.startswith('data: '):
                            line = line[len('data: '):]
                        # Last chunk, stream ends
                        if line.strip() == '[DONE]':
                            break
                        # Parse the JSON response
                        chunk = json.loads(line)
                        # Extract and print the content
                        content = chunk['choices'][0].get('delta',
                                                          {
            }).get('content')
                        print(content, end='')


# Run the asynchronous function
asyncio.run(stream_openai_response())
```

7.5 分词器API

我们的分词器API是HuggingFace风格的分词器的简单封装。它包含两个端点：

- **/tokenize** 对应于调用 `tokenizer.encode()`。
- **/detokenize** 对应于调用 `tokenizer.decode()`。

7.6 池化API

我们的池化API使用池化模型对输入提示进行编码，并返回相应的隐藏状态。

输入格式与嵌入API相同，但输出数据可以包含任意嵌套的列表，而不仅仅是一个一维的浮点数列表。

代码示例如： `openai_pooling_client.py`

```
# SPDX-License-Identifier: Apache-2.0
"""
Example online usage of Pooling API.

Run `vllm serve <model> --task <embed|classify|reward|score>`
to start up the server in vLLM.
"""
import argparse
import pprint

import requests
```

```python
import requests


def post_http_request(prompt: dict, api_url: str) -> requests.Response:
    headers = {
            "User-Agent": "Test Client"}
    response = requests.post(api_url, headers=headers, json=prompt)
    return response


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--host", type=str, default="localhost")
    parser.add_argument("--port", type=int, default=8000)
    parser.add_argument("--model",
                        type=str,
                        default="jason9693/Qwen2.5-1.5B-apeach")

    args = parser.parse_args()
    api_url = f"http://{
              args.host}:{
              args.port}/pooling"
    model_name = args.model

    # Input like Completions API
    prompt = {
            "model": model_name, "input": "vLLM is great!"}
    pooling_response = post_http_request(prompt=prompt, api_url=api_url)
    print("Pooling Response:")
    pprint.pprint(pooling_response.json())

    # Input like Chat API
    prompt = {

        "model":
        model_name,
        "messages": [{

            "role": "user",
            "content": [{

                "type": "text",
                "text": "vLLM is great!"
            }],
        }]
    }
    pooling_response = post_http_request(prompt=prompt, api_url=api_url)
    print("Pooling Response:")
```

```
    pprint.pprint(pooling_response.json())
```

7.7 评分API

我们的评分API可以应用交叉编码器模型或嵌入模型来预测句子对的评分。使用嵌入模型时，评分对应于每对嵌入的余弦相似度。通常，句子对的评分表示两句话之间的相似度，评分范围从0到1。

你可以在 **sbert.net** 上找到交叉编码器模型的文档。

代码示例： `openai_cross_encoder_score.py`

```
# SPDX-License-Identifier: Apache-2.0
"""
Example online usage of Score API.

Run `vllm serve <model> --task score` to start up the server in vLLM.
"""
import argparse
import pprint

import requests


def post_http_request(prompt: dict, api_url: str) -> requests.Response:
    headers = {
            "User-Agent": "Test Client"}
    response = requests.post(api_url, headers=headers, json=prompt)
    return response


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--host", type=str, default="localhost")
    parser.add_argument("--port", type=int, default=8000)
    parser.add_argument("--model", type=str, default="BAAI/bge-reranker-v2-m3")

    args = parser.parse_args()
    api_url = f"http://{
              args.host}:{
              args.port}/score"
    model_name = args.model

    text_1 = "What is the capital of Brazil?"
    text_2 = "The capital of Brazil is Brasilia."
    prompt = {
            "model": model_name, "text_1": text_1, "text_2": text_2}
    score_response = post_http_request(prompt=prompt, api_url=api_url)
    print("Prompt when text_1 and text_2 are both strings:")
```

```
    pprint.pprint(prompt)
    print("Score Response:")
    pprint.pprint(score_response.json())

    text_1 = "What is the capital of France?"
    text_2 = [
        "The capital of Brazil is Brasilia.", "The capital of France is Paris."
    ]
    prompt = {
            "model": model_name, "text_1": text_1, "text_2": text_2}
    score_response = post_http_request(prompt=prompt, api_url=api_url)
    print("Prompt when text_1 is string and text_2 is a list:")
    pprint.pprint(prompt)
    print("Score Response:")
    pprint.pprint(score_response.json())

    text_1 = [
        "What is the capital of Brazil?", "What is the capital of France?"
    ]
    text_2 = [
        "The capital of Brazil is Brasilia.", "The capital of France is Paris."
    ]
    prompt = {
            "model": model_name, "text_1": text_1, "text_2": text_2}
    score_response = post_http_request(prompt=prompt, api_url=api_url)
    print("Prompt when text_1 and text_2 are both lists:")
    pprint.pprint(prompt)
    print("Score Response:")
    pprint.pprint(score_response.json())
```

7.7.1 单次推理

你可以将一个字符串传递给 text_1 和 text_2，从而形成一个单独的句子对。

请求示例：

```
curl -X 'POST' \
  'http://127.0.0.1:8000/score' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "model": "BAAI/bge-reranker-v2-m3",
  "encoding_format": "float",
  "text_1": "What is the capital of France?",
  "text_2": "The capital of France is Paris."
}'
```

响应示例：

```
{

  "id": "score-request-id",
  "object": "list",
  "created": 693447,
  "model": "BAAI/bge-reranker-v2-m3",
  "data": [
    {

      "index": 0,
      "object": "score",
      "score": 1
    }
  ],
  "usage": {
            }
}
```

7.7.2 批量推理

你可以将一个字符串传递给 text_1，并将一个字符串列表传递给 text_2，从而形成多个句子对，其中每一对都由 text_1 和 text_2 中的每个字符串组成。总的句子对数是 len(text_2)。

请求示例：

```
curl -X 'POST' \
  'http://127.0.0.1:8000/score' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "model": "BAAI/bge-reranker-v2-m3",
  "text_1": "What is the capital of France?",
  "text_2": [
    "The capital of Brazil is Brasilia.",
    "The capital of France is Paris."
  ]
}'
```

回复实例：

```
{

  "id": "score-request-id",
  "object": "list",
  "created": 693570,
  "model": "BAAI/bge-reranker-v2-m3",
  "data": [
    {

      "index": 0,
      "object": "score",
      "score": 0.001094818115234375
    },
    {

      "index": 1,
      "object": "score",
      "score": 1
    }
  ],
  "usage": {
            }
}
```


批量推理

你可以将一个字符串列表传递给 `text_1` 和 `text_2`，从而形成多个句子对，其中每一对由 `text_1` 和 `text_2` 中对应的字符串组成（类似于 `zip()`）。句子对的总数是 `len(text_2)`。

请求示例：

```
curl -X 'POST' \
  'http://127.0.0.1:8000/score' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{

  "model": "BAAI/bge-reranker-v2-m3",
  "encoding_format": "float",
  "text_1": [
    "What is the capital of Brazil?",
    "What is the capital of France?"
  ],
  "text_2": [
    "The capital of Brazil is Brasilia.",
    "The capital of France is Paris."
  ]
}'
```

回复示例：


```
{

  "id": "score-request-id",
  "object": "list",
  "created": 693447,
  "model": "BAAI/bge-reranker-v2-m3",
  "data": [
    {

      "index": 0,
      "object": "score",
      "score": 1
    },
    {

      "index": 1,
      "object": "score",
      "score": 1
    }
  ],
  "usage": {
            }
}
```


7.7.3 额外参数

以下是支持的池化参数：

```
    additional_data: Optional[Any] = None
```

- **additional_data**: 可选的额外数据，类型为 `Any`，默认为 `None`。

此外，还支持以下额外参数：

- **priority**: `int` 类型，默认值为 `0`。
  描述：请求的优先级（数字越小表示处理越早；默认值为 `0`）。如果服务的模型不支持优先级调度，任何非 `0` 的优先级都会导致错误。

```
  priority: int = Field(
    default=0,
    description=(
        "The priority of the request (lower means earlier handling; "
        "default: 0). Any priority other than 0 will raise an error "
        "if the served model does not use priority scheduling."))
```

7.8 重新排序API

我们的重新排序（Re-rank）API可以应用嵌入模型或交叉编码器模型来预测一个查询与文档列表中每个文档之间的相关评分。通常，句子对的评分表示两个句子之间的相似度，评分范围为0到1。

你可以在 **sbert.net** 上找到交叉编码器模型的文档。

重新排序端点支持流行的重新排序模型，例如 **BAAI/bge-reranker-base** 和其他支持评分任务的模型。此外，`/rerank`、`/v1/rerank` 和 `/v2/rerank` 端点兼容 Jina AI 的重新排序 API 接口和 Cohere 的重新排序 API 接口，以确保与流行的开源工具兼容。

代码示例：`jinaai_rerank_client.py`

```python
# SPDX-License-Identifier: Apache-2.0
"""
Example of using the OpenAI entrypoint's rerank API which is compatible with
Jina and Cohere https://jina.ai/reranker

run: vllm serve BAAI/bge-reranker-base
"""
import json

import requests

url = "http://127.0.0.1:8000/rerank"

headers = {
            "accept": "application/json", "Content-Type": "application/json"}

data = {

    "model":
    "BAAI/bge-reranker-base",
    "query":
    "What is the capital of France?",
    "documents": [
        "The capital of Brazil is Brasilia.",
        "The capital of France is Paris.", "Horses and cows are both animals"
    ]
}
response = requests.post(url, headers=headers, json=data)

# Check the response
if response.status_code == 200:
    print("Request successful!")
    print(json.dumps(response.json(), indent=2))
else:
    print(f"Request failed with status code: {
              response.status_code}")
    print(response.text)
```

7.8.1 示例请求

请注意，`top_n` 请求参数是可选的，默认值为 `documents` 字段的长度。结果文档将按相关性排序，`index` 属性可用于确定原始顺序。

请求示例：

```
curl -X 'POST' \
  'http://127.0.0.1:8000/v1/rerank' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "model": "BAAI/bge-reranker-base",
  "query": "What is the capital of France?",
  "documents": [
    "The capital of Brazil is Brasilia.",
    "The capital of France is Paris.",
    "Horses and cows are both animals"
  ]
}'
```

回复示例：

```
{

  "id": "rerank-fae51b2b664d4ed38f5969b612edff77",
  "model": "BAAI/bge-reranker-base",
  "usage": {

    "total_tokens": 56
  },
  "results": [
    {

      "index": 1,
      "document": {

        "text": "The capital of France is Paris."
      },
      "relevance_score": 0.99853515625
    },
    {

      "index": 0,
      "document": {

        "text": "The capital of Brazil is Brasilia."
      },
      "relevance_score": 0.0005860328674316406
    }
  ]
}
```

7.8.2 额外参数

以下是支持的池化参数：

- **additional_data**: 可选的额外数据，类型为 `Any`，默认为 `None`。

此外，还支持以下额外参数：

- **priority**: `int` 类型，默认值为 `0`。
  描述：请求的优先级（数字越小表示处理越早；默认值为 `0`）。如果服务的模型不支持优先级调度，任何非 `0` 的优先级都会导致错误。